A Comparative Analysis of BFS and A* Pathfinding Visualization in Grid-Based Mazes

Hemanthi Deepak Kannan,

Student, Department of Computer Science and Engineering with Artificial Intelligence, Sathyabama Institute of Science and Technology, Jeppiaar Nagar, Rajiv Gandhi Salai, Chennai – 600 119.

Abstract

Pathfinding is a fundamental problem in artificial intelligence, with applications in game development, robotics, and autonomous systems. This paper presents a comparative study of Breadth-First Search (BFS) and A* pathfinding algorithms using a real-time visualization built with Pygame. Both algorithms were implemented on an identical 10×10 grid-based maze, and their performance was evaluated using quantitative metrics such as number of nodes explored, execution time, and path optimality. Experimental results show that while BFS guarantees optimal paths, it explores significantly more nodes compared to A*. In contrast, A* efficiently guides the search using a heuristic, resulting in fewer explored nodes and faster completion times. The visualization highlights these differences clearly and demonstrates the practical advantages of heuristic-based search algorithms in constrained environments.

---

1. Introduction

Pathfinding algorithms play a crucial role in artificial intelligence, particularly in applications such as video games, navigation systems, and robotics. Efficient path planning is essential for real-time systems where computational resources are limited. Among the commonly used algorithms, Breadth-First Search (BFS) and A* represent two contrasting approaches: uninformed and informed search.

BFS explores all possible paths uniformly without domain-specific knowledge, while A* incorporates heuristic information to guide the search toward the goal. This project aims to visually and quantitatively compare these two algorithms in a controlled grid-based maze environment, making their behavioural differences intuitive and measurable.

2. Methodology

2.1 Environment Setup

A fixed 10×10 grid-based maze was designed, where each cell represents either a free space or a wall. The start position was fixed at the top-left corner, and the goal position was fixed at the bottom-right corner. Movement was restricted to four directions: up, down, left, and right.

2.2 Algorithm Implementation

- **Breadth-First Search (BFS):**

  BFS was implemented using a queue-based approach, exploring neighbouring nodes level by level. It guarantees the shortest path in unweighted graphs but does not prioritize direction.

- **A* Search:**

  A* was implemented using a priority queue and the Manhattan distance heuristic. The heuristic estimates the cost from the current node to the goal, allowing the algorithm to prioritize promising paths.

Both algorithms were implemented as Python generators to enable step-by-step visualization.

2.3 Visualization and Metrics

The visualization was developed using the Pygame library. During execution:

- Visited nodes were highlighted in real time.
- The final path was displayed once the goal was reached.
- Performance metrics were recorded:

  i. Number of nodes explored
  ii. Time taken to find the solution (milliseconds)
  iii. Final path length

3. Results

The following table summarizes the observed results for both algorithms on the same maze:

Algorithm	Nodes Explored	Time (ms)	Path Length
BFS	58	3864	19
A*	19	1265	19

Table 3.1: Summary of Performance Metrics

Both algorithms produced an optimal path of equal length. However, A* explored approximately 67% fewer nodes and completed the search in significantly less time.

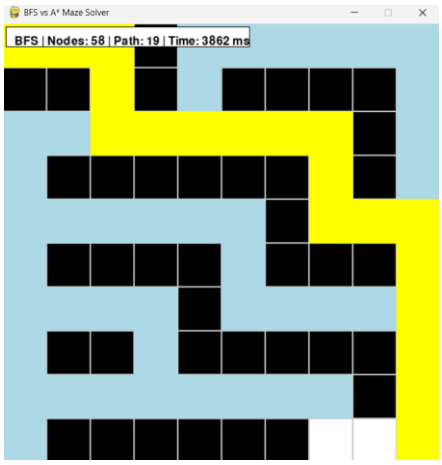

Fig 3.1: BFS exploration Pattern

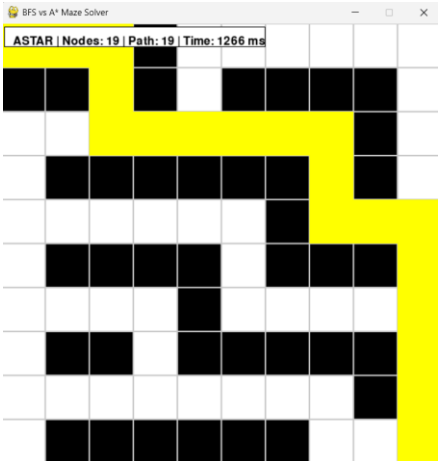

Fig 3.2: A exploration Pattern*

As shown in Figure 3.1, BFS explores a significantly larger portion of the maze, whereas A* (Figure 3.2) focuses the search toward the goal.

4. Discussion

The results clearly demonstrate the efficiency benefits of heuristic-guided search. BFS performs a blind exploration of the maze, leading to a large number of unnecessary node visits. While this guarantees optimality, it becomes computationally expensive as maze size increases. A*, on the other hand, uses heuristic guidance to focus the search toward the goal, significantly reducing exploration while maintaining optimality. The visualization further emphasizes this difference, as BFS expands uniformly outward, whereas A* progresses in a directed manner. One limitation of this study is the use of a fixed maze size. Larger or dynamically changing environments could further highlight the scalability advantages of A*.

5. Conclusion

This paper presented a comparative visualization and performance analysis of BFS and A* pathfinding algorithms in a grid-based maze. While BFS ensures optimal solutions through exhaustive exploration, A* achieves the same optimality with greater efficiency by incorporating heuristic information. The results confirm that A* is better suited for real-time and large-scale applications where computational efficiency is critical. Future work may include dynamic obstacle handling, alternative heuristics, and larger grid environments.

References

1. Norvig, P. and Russell, S.J., 1995. Artificial intelligence a modern approach.

2. Hart, P.E., Nilsson, N.J. and Raphael, B., 1968. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, *4*(2), pp.100-107.

3. Pygame Documentation. https://www.pygame.org/docs/